

ByteBlower Docs, Distilled

Oct 8, 2026 · @sdfkmsldfmsldfm

What ByteBlower is

ByteBlower is a traffic generator and analyzer from Excentis: a 1U server generates and measures traffic, and a client computer drives it. The documentation is spread over four sites, distilled here from about 370 pages crawled on 8 October 2026.

The server chassis holds three isolated systems ([components](#)):

- **Traffic generation software:** owns the traffic interfaces and most of the CPU. Updates need no internet on the server.
- **Host operating system:** owns the management interfaces (`man0` , `man1`), reachable over SSH. Default login is `root` / `excentis` .
- **IPMI:** optional remote hardware control (power, boot).

There is no software limit on the number of clients, and sharing one server among users is normal.

Interface	What it is	Use it when
ByteBlower GUI	Desktop app with wizards, project files (<code>.bbp</code>) and an HTML/PDF reporting engine	Building and running tests by hand
ByteBlower-CLT	Command-line runner for the same project files	Scheduling GUI projects from scripts or CI
Test Framework	Python library and CLI driven by a JSON scenario file, with its own HTML/JSON/JUnit reports	Automating standard tests without writing much code
Python API (<code>byteblower11</code>)	Low-level object API, same building blocks as the GUI, no reporting engine	Full control and raw results
Tcl API	Same low-level model in Tcl, plus a higher-layer <code>ByteBlowerHL</code> package	Existing Tcl automation

Interface	What it is	Use it when
OpenMetrics exporter	The GUI serves live metrics on TCP 8123 for Prometheus	Real-time graphs in Grafana, or scenarios over 100 flows

Model differences that recur throughout the docs:

Model	Notes
1x00	Oldest series. NTP for time. Cannot measure latency at high speed.
2100	Hardware-accelerated. PTP supported and preferred.
3100 / 3200	Software traffic generation on Intel x710 NICs. NTP for time. NIC firmware fix may be needed.
4100	Hardware-accelerated. PTP supported and preferred.
5100	Up to 100 Gbps. NTP only. May sample received packets for latency. Upgrades only through the GUI.

Core concepts

Every test is flows between ByteBlower Ports, and every interface (GUI, Test Framework, APIs) uses the same vocabulary ([basic test units](#)).

Term	Meaning
Port	A small virtual host with its own protocol stack, docked to one server interface. Needs a MAC; gets IPv4 (fixed or DHCP) and/or IPv6 (fixed, SLAAC or DHCPv6). Answers ping once it has an address.
Interface	Physical attachment point. Nontrunk sits on the server itself (<code>nontrunk-1</code>). Trunk sits on a ByteBlower switch and shares a server connector (<code>trunk-1-20</code>).

Term	Meaning
Frame	A layer-2 Ethernet II frame you define, 60 to 10000 bytes excluding FCS.
Frame Blasting	Stateless traffic: one port injects frames at a set rate, the destination counts them. Gives exact, repeatable loss and latency.
TCP flow	Stateful traffic. Always HTTP over TCP: an HTTP client on one port, an HTTP server on the other. Self-limits to available bandwidth.
Flow	One or more frames tracked as one result, from one source to one or more destinations.
Scenario	Flows with start times and durations. The runnable unit; one report per run.
Batch	Scenarios run one after another.
Trigger	A receive-side counter selected by a BPF filter. This is how received frames are attributed to a flow.
Endpoint	An app on a phone or laptop acting as a port. Controlled through a Meeting Point, never directly.

Measurements and how they work:

- **Loss:** frames sent minus frames matched by the destination trigger. If two flows have near-identical frames the filters collide and loss goes negative.
- **Latency:** the sender overwrites the last 8 bytes of the payload with a timestamp; the receiver subtracts it from its own clock. Sender and receiver on different machines need synchronized clocks.
- **Out-of-sequence:** an incrementing frame number in the payload. Reports reordering, not which frames were lost.
- **TCP:** goodput (application rate), throughput (including headers and retransmissions), round-trip time and transmit window.
- **NAT:** when a port is behind NAT, ByteBlower sends about 10 discovery frames per second for up to 20 seconds to learn the public address and port, then counts on those ([NAT](#)).

TCP direction follows the HTTP method. GET makes the destination initiate and download; PUT makes the source initiate and upload. AUTO picks PUT when the source is behind NAT, otherwise GET ([TCP timing](#)).

Bitrates depend on what you count. Each frame costs 24 extra bytes on the wire (preamble 7, SFD 1, FCS 4, gap 12), so 60-byte frames at 1 Gbps line rate are only 714 Mbps of frame bitrate ([link speed](#)).

Server setup and maintenance

Server administration runs through `byteblower-configurator` over SSH or a keyboard and screen; the management IP defaults to DHCP on `man0`.

Task	How
Find the management IP	Server 2.6.0 and later show it on the login screen. Earlier: log in and run <code>ifconfig man0</code> .
Set a static management IP	Configurator: System Configuration, Network Configuration, select <code>man0</code> , Connectivity, static (article).
Change the root password	<code>passwd</code> , or the configurator. A lost password means reinstalling.
Set time	Configurator: Time Configuration. NTP on 1x00, 3x00, 5x00. PTP on 2100 and 4100, using the PTP cable shipped with the unit; NTP there only corrects at startup (article).
Regroup trunk interfaces	Possible since server 2.14 for daisy-chained and multi-trunk switches. Contact support to convert a layout.
Configure IPMI	<code>bmc-config --checkout --section=Lan_Conf --filename=ipmi-network.conf</code> , edit, then <code>bmc-config --commit --filename=ipmi-network.conf</code> .
Upgrade	GUI upgrade wizard. The client PC needs internet and SSH to the server; the server does not. About 10 minutes with reboots. The only method on the 5100.

Task	How
Downgrade	<code>excentux-update</code> , pick the version, reboot. Not for the 5100. Talk to support first: forward compatibility is limited.
Check version	<code>byteblower-get-version</code> , or the GUI Server view.

Firewall ports toward the server ([remote access](#)):

TCP port	Use
22	SSH, also used by the GUI upgrade wizard
9002	GUI or API to the ByteBlower server
9100	Endpoint heartbeat to the Meeting Point
9101	GUI or API to the Meeting Point

Ports 9100 and 9101 only matter with a Meeting Point license. The server has no firewall of its own.

First check after installation is the Quick self-test: cable two trunk ports back to back, open the example `.bbp` , add the server by IP, dock both ports, run. The report should read 100% of frames received ([self-test](#)).

For one-way latency across machines, PTP on two 4100 servers stayed within a couple of microseconds. Building your own PTP master on an Intel NUC with `linuxptp` is documented step by step ([time sync](#)).

Endpoint and Meeting Point

The Endpoint is an app that turns a phone, laptop or NUC into a traffic port; the Meeting Point is a daemon on the ByteBlower server that every Endpoint registers with and that relays all commands and results.

A test runs in four states: Contacting, Registered (idle), Armed (scenario received), Running. By default the Endpoint goes silent during a run and uploads results afterwards. Since 2.19.0 Heartbeat Mode keeps it in contact so a run can be cancelled remotely; in the API that is `ScenarioHeartbeatIntervalSet` , in nanoseconds.

The Endpoint must be able to reach the Meeting Point. Three topologies are documented ([connection](#)):

1. Through the server's management interface, usually via the gateway's WAN side.
2. Through a spare server interface wired into the test network, when management is fully separate.
3. Through a second management interface plugged into a LAN port of the device under test, when it has no uplink.

Setup:

- Install from the app stores, or download for Windows, macOS and Linux. Android without Google Play: sideload the APK with `adb install ByteBlower.apk`.
- Enter the server's IPv4 address or hostname as the Meeting Point.
- Headless: `byteblower-wireless-endpoint [--management <nic>] [--traffic <nic>] <meetingpoint>`. A systemd unit and a Windows Task Scheduler recipe are documented for autostart ([autostart](#)).
- In the GUI the server appears twice: once as wired, once as Meeting Point. Dock the wireless port on the device listed under the Meeting Point.

What an Endpoint cannot do, compared with a server port ([limits](#)):

- One frame per Frame Blasting template.
- A UDP port can be used once for transmit and once for receive per scenario.
- No out-of-sequence detection.
- As a destination it reports only average latency, not the distribution.
- Latency is best-effort: the clock syncs to the Meeting Point at registration and is not re-synced during a test.
- iOS reports SSID and BSSID but never RSSI, channel or Tx rate. Android and macOS need location permission for Wi-Fi statistics.

L4S needs server, GUI and Endpoint at 2.22.0 or later. On Linux it needs the L4S team's custom kernel with `tcp_prague` and `sch_dualpi2`; macOS and iOS have Accurate ECN only ([L4S](#)).

The "Golden Client" is a reference Endpoint on an Intel NUC with Debian, PTP via `linuxptp` and `chrony`. Wired latency accuracy is reported as well below 1 ms ([Golden Client](#)).

Test Framework

The Test Framework is the fastest route to automation: describe ports and flows in a JSON file, run one command, get HTML, JSON and JUnit XML reports with pass/fail. The docs show version 1.4.2, dated 2026/09/21, for Python 3.7 to 3.11 ([docs](#)).

```
python3 -m venv --clear .venv && . ./venv/bin/activate
pip install -U byteblower-test-framework
byteblower-test-framework --config-file my_test_config.json --report-path
reports
```

Every command takes the same three options: `--config-file`, `--config-path` (default `.`) and `--report-path` (default `.`).

Package	Command	Default config file	What it tests
<code>byteblower-test-framework</code>	<code>byteblower-test-framework</code>	<code>byteblower_test_framework.json</code>	Any mix of UDP and TCP flows
<code>byteblower-test-cases-low-latency</code>	<code>byteblower-test-cases-low-latency</code>	<code>low_latency.json</code>	Low Latency DOCSIS and L4S, with voice, video, gaming and conference traffic
<code>byteblower-test-cases-rfc-2544</code>	<code>byteblower-test-cases-rfc-2544-throughput</code>	<code>rfc_2544.json</code>	Highest lossless rate per frame size
<code>byteblower-test-cases-tr-398</code>	<code>byteblower-test-cases-tr-398-airtime-fairness</code>	<code>tr_398.json</code>	Wi-Fi airtime fairness across three stations
<code>byteblower-test-cases-docsis-atp</code>	<code>byteblower-test-cases-docsis-atp-mulpi-v4-0-11-04-40-part1</code>	<code>part1.json</code>	DOCSIS 4.0 PIE AQM with and without low-latency traffic

The same run from Python: `from byteblower_test_framework import run`, then `run(test_config, report_path=..., report_prefix=...)` with the JSON content as a dict.

Scenario file

Required root keys are `server`, `ports` and `flows`. Optional: `meeting_point`, `maximum_run_time` (seconds), `report` (`html`, `json`, `junit_xml` booleans) and `enable_scouting_flows`.

```

{
  "$schema": "https://api.byteblower.com/test-framework/json/cli-config-
schema.json",
  "server": "byteblower-server.example.com.",
  "ports": [
    {
      "name": "NSI",
      "port_group": ["classic_nsi"],
      "interface": "trunk-1-3",
      "ipv4": "192.168.1.2",
      "netmask": "255.255.255.0",
      "gateway": "192.168.1.254"
    },
    {
      "name": "CPE",
      "port_group": ["classic_cpe"],
      "interface": "trunk-1-4",
      "ipv4": "dhcp",
      "nat": true
    }
  ],
  "flows": [
    {
      "name": "Downstream UDP",
      "type": "frame_blasting",
      "source": {"port_group": ["classic_nsi"]},
      "destination": {"port_group": ["classic_cpe"]},
      "frame_size": 60,
      "bitrate": 850000000.0,
      "analysis": {
        "latency": true,
        "max_loss_percentage": 1,
        "max_threshold_latency": 20
      }
    },
    {
      "name": "Bidirectional HTTP",
      "type": "http",
      "source": {"port_group": ["classic_nsi"]},
      "destination": {"port_group": ["classic_cpe"]},
      "maximum_bitrate": 200000000.0,
      "initial_time_to_wait": 2.0,
      "duration": 40.0,

```

```

        "receive_window_scaling": 12,
        "add_reverse_direction": true
    }
],
"report": {"html": true, "json": true, "junit_xml": false},
"enable_scouting_flows": true,
"maximum_run_time": 30.0
}

```

To use an Endpoint, add a root `meeting_point` and replace the port's `interface` with `"uuid": "<endpoint uuid>", "ipv4": true`.

Port key	Values	Default
<code>port_group</code>	List of names; flows reference groups, and one flow is created per source and destination pair	
<code>interface</code>	<code>nontrunk-N</code> or <code>trunk-N-M</code> . Server ports only	
<code>uuid</code>	Endpoint only; excludes <code>interface</code>	
<code>ipv4</code>	Address or <code>"dhcp"</code> ; <code>true</code> on an Endpoint	<code>"dhcp"</code>
<code>ipv6</code>	<code>"dhcp"</code> , <code>"slaac"</code> or a list of <code>{address, prefix_length}</code> ; <code>true</code> on an Endpoint	unset
<code>netmask</code> , <code>gateway</code>	Static addressing only	<code>255.255.255.0</code> , none
<code>nat</code>	Port is behind IPv4 NAT; triggers discovery	<code>false</code>
<code>firewall</code>	Port is behind an IPv6 firewall	<code>false</code>
<code>mac</code>	Server ports only	generated
<code>vlan</code>	List, outer first, of <code>{id, protocol_id, priority, drop_eligible}</code>	
<code>capture</code>	<code>{enabled, filter, filename}</code> ; receive side only	off

Flow key	Applies to	Meaning	Default
<code>type</code>	all	<code>frame_blasting</code> or <code>http</code>	
<code>dscp</code> , <code>ecn</code>	all	Integer or hex string	0
<code>initial_time_to_wait</code>	all	Seconds	0
<code>add_reverse_direction</code>	all	Adds the mirrored flow	<code>false</code>
<code>bitrate</code> or <code>frame_rate</code>	UDP	Bits per second excluding VLAN bytes, or frames per second; pick one	100 fps
<code>frame_size</code>	UDP	Bytes without CRC, minimum 42	1024
<code>duration</code> or <code>number_of_frames</code>	UDP	Seconds, or a frame count	scenario run time
<code>nat_keep_alive</code>	UDP	Keeps the NAT entry open	<code>false</code>
<code>analysis.max_loss_percentage</code>	UDP	Fail above this loss	1.0
<code>analysis.latency</code>	UDP	Adds latency over time, CDF and CCDF	<code>false</code>
<code>analysis.max_threshold_latency</code> , <code>quantile</code>	UDP	Latency at the quantile must stay under the threshold, in ms	5.0, 99.9
<code>duration</code> or <code>request_size</code>	HTTP	Seconds, or bytes; pick one	
<code>maximum_bitrate</code>	HTTP	Bits per second	unlimited
<code>receive_window_scaling</code>	HTTP	0 to 12	
<code>slow_start_threshold</code>	HTTP	Integer; unit not stated	

Flow key	Applies to	Meaning	Default
<code>enable_l4s</code>	HTTP	TCP Prague with CE-mark counting; needs 2.22.0 everywhere	<code>false</code>

HTTP flows are measured but never fail a test. The overall result is FAIL if any flow fails.

`rate_limit` (bytes per second) and `napt_keep_alive` are deprecated in favour of `maximum_bitrate` and `nat_keep_alive`.

Test cases

- **Low Latency** adds flow types `dynamic_frame_blasting`, `l4s_frame_blasting`, `voice` (G.711, scored by MOS, pass at 4 or above), `video`, `gaming` (latency threshold 5 ms) and `conference`. Video, dynamic and L4S frame blasting do not run on Endpoints.
- **RFC 2544** takes `source`, `destination` and optional `frame_configs` (`size`, `initial_bitrate`, `tolerated_frame_loss`, `accuracy`, `expected_bitrate`). It searches the rate per frame size by halving and doubling, then bisecting. Default sizes are 60, 124, 252, 508, 1020, 1276 and 1514 bytes. It fails when any size lands below `expected_bitrate`.
- **TR-398 Airtime Fairness** takes `dut` and exactly three `wlan_stations` as Endpoints. Each station must reach more than 45% of its own TCP maximum under paired UDP load, and the pair must beat the TR-398 expected total. Moving stations between runs is manual.
- **DOCSIS 4.0 ATP LL-04.40 Part 1** takes `nsi` and `cpe`. It runs two procedures of two 20-second classic flows, the second alongside a 40-second low-latency flow. The pass thresholds are not published in the docs.

Python classes

The library behind the CLI lives in `byteblower_test_framework`. Release order is scenario and flows, then ports, then server.

Module	Classes
run	Scenario: add_flow, add_report, add_capture, run(maximum_run_time=...), report(), release()
host	Server(ip_or_host), MeetingPoint(ip_or_host)
endpoint	IPv4Port, IPv6Port, NatDiscoveryIPv4Port, NatDiscoveryIPv6Port, IPv4Endpoint, IPv6Endpoint
traffic	FrameBlastingFlow, HTTPFlow, VoiceFlow, VideoFlow, GamingFlow; frames IPv4Frame, IPv6Frame, MobileFrame, Imix
analysis	FrameLossAnalyser, LatencyFrameLossAnalyser, LatencyCDFFrameLossAnalyser, VoiceAnalyser, HttpAnalyser, L4SHttAnalyser, BufferAnalyser, PortCapture
report	ByteBlowerHtmlReport, ByteBlowerJsonReport, ByteBlowerUnitTestReport

The published docs contain no hand-written `Scenario` script, and the method that attaches an analyser to a flow is not on the reference pages. Start from the JSON route or the package source.

Python API (byteblower11)

The low-level API is one singleton from which you create servers, ports, streams and receivers, then poll result objects; you build the frame bytes yourself, normally with scapy. Install with `pip install byteblower11 scapy` ([reference](#), [examples](#)).

```

ByteBlower.InstanceGet()
  ServerAdd(host, port=9002) -> ByteBlowerServer
  PortCreate('trunk-1-13') -> ByteBlowerPort
  Layer2EthIISet() -> MacSet
  Layer25VlanAdd() -> IDSet          (call again to stack; first is
outermost)
  Layer3IPv4Set() -> IpSet, NetmaskSet, GatewaySet |
ProtocolDhcpGet().Perform(); Resolve(ip)
  Layer3IPv6Set() -> IpManualAdd('addr/len') |
StatelessAutoconfiguration() | ProtocolDhcpGet().Perform()
  TxStreamAdd() -> Stream: NumberOfFramesSet, InterFrameGapSet,
Start, Stop
  FrameAdd() -> Frame: BytesSet(hex), FrameTagTimeGet(),
FrameTagSequenceGet()
  RxTriggerBasicAdd() counts frames matching a BPF filter
  RxLatencyBasicAdd() latency min, average, max, jitter
  RxLatencyDistributionAdd() latency histogram
  RxOutOfSequenceBasicAdd() reordering
  RxCaptureBasicAdd() pcap capture
  ProtocolHttpServerAdd() / ProtocolHttpClientAdd()
MeetingPointAdd(host, port=9101) -> MeetingPoint
  DeviceGet(uuid) -> WirelessEndpoint: Lock, Prepare, Start, ResultGet
  TxStreamAdd() -> StreamMobile; RxTriggerBasicAdd() ->
TriggerBasicMobile; ProtocolHttpClientAdd()

```

UDP frame blasting between two ports, condensed from the official `back2back/ipv4.py`:

```

from time import sleep
from byteblowerll.byteblower import ByteBlower
from scapy.layers.inet import UDP, IP, Ether
from scapy.all import Raw

bb = ByteBlower.InstanceGet()
server = bb.ServerAdd('byteblower-server.example.com')

def make_port(interface, mac):
    port = server.PortCreate(interface)
    port.Layer2EthIISet().MacSet(mac)
    port.Layer3IPv4Set().ProtocolDhcpGet().Perform()
    return port

tx = make_port('trunk-1-13', '00:bb:01:00:00:01')

```

```

rx = make_port('trunk-1-14', '00:bb:01:00:00:02')
src_ip = tx.Layer3IPv4Get().IpGet()
dst_ip = rx.Layer3IPv4Get().IpGet()
dst_mac = tx.Layer3IPv4Get().Resolve(dst_ip)      # peer MAC, or the
gateway's when off-subnet

frame_size, n_frames, gap_ns, udp_port = 512, 10000, 1000000, 4096
scapy_frame = (Ether(src=tx.Layer2EthIIGet().MacGet(), dst=dst_mac)
               / IP(src=src_ip, dst=dst_ip) / UDP(sport=udp_port,
dport=udp_port)
               / Raw(b'a' * (frame_size - 42)))

stream = tx.TxStreamAdd()
stream.NumberOfFramesSet(n_frames)
stream.InterFrameGapSet(gap_ns)                  # ns between frame starts:
1000 fps
stream.FrameAdd().BytesSet(''.join(format(b, '02x') for b in
bytearray(bytes(scapy_frame))))

trigger = rx.RxTriggerBasicAdd()
trigger.FilterSet('ip dst {} and udp port {}'.format(dst_ip, udp_port))
trigger.ResultClear()                            # triggers count from
creation

stream.Start()
sleep(n_frames * gap_ns / 1e9 + 1)               # one extra second for
frames in flight

tx_result, rx_result = stream.ResultGet(), trigger.ResultGet()
tx_result.Refresh(); rx_result.Refresh()
print(tx_result.PacketCountGet(), rx_result.PacketCountGet())

server.PortDestroy(tx); server.PortDestroy(rx)
bb.ServerRemove(server)

```

Variations on that script:

To measure	Change
Latency	<code>frame.FrameTagTimeGet().Enable(True)</code> and receive with <code>rx.RxLatencyBasicAdd();</code> read <code>LatencyAverageGet()</code> , <code>LatencyMinimumGet()</code> , <code>LatencyMaximumGet()</code> , <code>JitterGet()</code> in ns
Reordering	<code>frame.FrameTagSequenceGet().Enable(True)</code> and <code>rx.RxOutOfSequenceBasicAdd();</code> read <code>PacketCountOutOfSequenceGet()</code>
IPv6	<code>Layer3IPv6Set()</code> , scapy <code>IPv6</code> , filter <code>ip6 dst ...</code> and <code>udp port ...</code>
TCP	<code>http_server = p1.ProtocolHttpServerAdd();</code> <code>http_server.Start().</code> On the client: <code>RemoteAddressSet</code> , <code>RemotePortSet</code> , <code>HttpMethodSet</code> , <code>RequestDurationSet(ns)</code> or <code>RequestSizeSet(bytes)</code> , <code>RequestStart()</code> , <code>WaitUntilConnected(ns)</code> , <code>WaitUntilFinished(ns)</code> . Results via <code>HttpSessionInfoGet().ResultGet()</code>
Many flows	Start ports together with <code>server.PortsStart(ByteBlowerPortList);</code> refresh everything in one round trip with <code>bb.ResultsRefresh(AbstractRefreshableResultList)</code>
Over time	<code>ResultHistoryGet()</code> , <code>Refresh()</code> each second, then <code>IntervalLatestGet()</code> or iterate <code>IntervalGet()</code>

Results come in two forms. `ResultGet()` is a cumulative snapshot. `ResultHistoryGet()` is a fixed-length ring of interval and cumulative samples, one per second by default, whose last sample is still open. Both are local copies until you call `Refresh()`.

Units seen in the official examples:

Quantity	Unit
Durations, inter-frame gap, timeouts, sampling interval, latency, jitter	nanoseconds, integers
Timestamps	POSIX time in nanoseconds
Request size, byte counters, congestion window	bytes
<code>Frame.BytesSet</code>	hex string of the whole Ethernet frame, no CRC
<code>DataRate</code>	<code>bitrate()</code> in bits per second, <code>MbpsGet()</code> , <code>toString()</code>

An Endpoint runs a prepared scenario instead of live commands:

1. `endpoint = meetingpoint.DeviceGet(uuid)`, then `endpoint.Lock(True)`.
2. Configure. Transmit uses `FrameMobile.PayloadSet` (UDP payload only) with `DestinationAddressSet`, `DestinationPortSet`, `SourcePortSet` on the stream. Receive uses `FilterUdpSourcePortSet`, `FilterUdpDestinationPortSet`, `FilterSourceAddressSet` and `DurationSet(ns)` instead of BPF.
3. `endpoint.Prepare()`, then `start = endpoint.Start()`, which returns a future start time. Sleep until `start - meetingpoint.TimestampGet()` has passed, then start the wired side.
4. After the run wait a heartbeat or two, call `endpoint.ResultGet()`, refresh the individual results, then `endpoint.Lock(False)`.

Rules that bite:

- `ServerRemove`, `MeetingPointRemove` and `DestroyInstance` invalidate every object under them; touching a stale handle can crash the process.
- A receiver without `FilterSet` counts every frame on the port.
- Read latency getters only when `PacketCountGet()` is non-zero.
- Feature support varies by server and device: check `port.CapabilityIsSupported(name)` before relying on a feature.
- `ServerAdd` raises `ByteBlowerServerUnreachable` or `ByteBlowerServerIncompatible`; `MeetingPointAdd` raises `MeetingPointUnreachable`.

The reference is one very long page and only its first part could be read. Everything after `ByteBlowerLicense` is covered here by method names and by usage in the official examples, not by the reference text. The Tcl class pages describe the same objects in full and are the fallback for parameters and defaults.

Tcl API

The Tcl API exposes the same objects as Python with dotted method names, and its per-class pages are the most complete description of the low-level model. The docs show version 2.22.2 ([reference](#)).

Package	Namespace	Purpose
<code>ByteBlower</code>		Lower-layer object API
<code>ByteBlowerHL</code>	<code>excentis::ByteBlower</code>	Higher-layer scenarios as nested lists. Frame blasting only.
<code>excentis_basic</code>	<code>excentis::basic</code>	Frame builders, checksums, address helpers

Names map mechanically. Python `port.Layer3IPv4Set()` is Tcl `$port Layer3.IPv4.Set`; `TxStreamAdd` is `Tx.Stream.Add`; `RxLatencyBasicAdd` is `Rx.Latency.Basic.Add`; `ResultHistoryGet` is `Result.History.Get`. Durations are nanoseconds but accept suffixes: `$stream InterFrameGap.Set 100ms`. Every object has `Description.Get` and `Destructor`; `$bbServer Destructor` cleans up a whole test.

```
package require ByteBlower
set bb [ ByteBlower Instance.Get ]
set server [ $bb Server.Add "byteblower.lab.company.com" ]
set port [ $server Port.Create "trunk-1-1" ]
[ $port Layer2.EthII.Set ] Mac.Set "00ff1c000001"
[ [ $port Layer3.IPv4.Set ] Protocol.Dhcp.Get ] Perform

set trigger [ $port Rx.Trigger.Basic.Add ]
$trigger Filter.Set "udp port 2002"
$trigger Result.Clear
set result [ $trigger Result.Get ]
$result Refresh
puts [ $result PacketCount.Get ]
```

The higher-layer API runs a whole scenario in one call. A flow is a list of `-tx` and `-rx` components, and `ExecuteScenario` sets up, runs, collects and tears down:

```
set flow [ list -tx [ list -port $srcPort -frame [ list -bytes $frameBytes ] \
    -numberofframes 1000 -interframegap 1000000 ] \
    -rx [ list -port $dstPort -trigger [ list -type basic -
filterFormat bpf \
    -filter "udp dst port 2002" ] ] ]
set result [ ::excentis::ByteBlower::ExecuteScenario [ list $flow ] ]
# {-tx {NrOfFramesSent 1000 ...} -rx {NrOfFrames 1000}}
```

Higher-layer proc	Returns
<code>ExecuteScenario</code>	Full counters per flow. Options <code>-finaltimetowait</code> (ms, default 5000), <code>-extended</code>
<code>ExecuteScenarioRT</code>	Same, with a callback every <code>-updateinterval</code> ms (default 1000)
<code>FlowLatency</code>	Per flow: frames sent, and received, min, average, max latency and jitter in ns
<code>FlowLossRate</code>	Loss per RFC 1242 as a percentage, counts, or per flow
<code>FlowOutOfsequence</code>	Frames sent, received, and out of sequence
<code>NatDevice.IP.Get</code>	Public IP and UDP port of a NAT device
<code>PPPoE.Setup</code> , <code>PPPoE.Start</code> , <code>PPPoE.MultiPPPoESessions.*</code>	PPPoE session helpers

Defaults and limits stated on the Tcl class pages, which describe the same server objects Python uses:

Item	Value
Frame bytes	60 to 8192 bytes, no CRC; can be changed during transmission

Item	Value
<code>NumberOfFrames</code>	<code>-1</code> means continuous
MDL (largest frame a port sends)	Default 1514; maximum typically 9014 on nontrunk, 9010 on trunk
Result history	One sample per second, 6 kept; refresh at least every 5 seconds or lose samples. Configurable since 2.3.0
DHCP	Initial timeout 1 s, 5 retries, release on port destruction
ARP cache	Entries valid 120 s
HTTP receive window	Initial 65535 bytes, scaling on, scale value 3 (range 0 to 8)
HTTP history	500 ms samples, 10 kept
Dual stack	Not on one port. Create two ports with the same layer 2 settings
<code>Filter.Set</code> on basic receivers	Resets counters and invalidates history

Migrating old scripts ([guide](#)):

- `Counters.Get` name-value lists became result objects: `[ $trigger Result.Get ] PacketCount.Get`.
- Static calls such as `ByteBlower Server.Add` are deprecated; call them on `[ ByteBlower Instance.Get ]`.
- Typed adders replaced generic ones: `Rx.Trigger.Add` became `Rx.Trigger.Basic.Add`, `Layer3.Set` became `Layer3.Ipv4.Set`.
- Frame tags moved: `TimeTag.Enable` became `FrameTag.Time.Get`.
- Size modifiers moved from stream to frame and now apply per frame, which changes the size sequence on multi-frame streams.
- Poll `$httpClient Finished.Get` instead of TCP status strings. The `serverDefault` congestion algorithm is gone; name one.
- IGMP and MLD put the version in the method name: `Session.V2.Add`.

Known bugs listed by the docs: HTTP requests cannot be started through `Schedules.Start` (use `Ports.Start`); `Logging.File.Name.Set` fails silently on a bad path; `ByteBlowerServer::Update` is not implemented; the Telnet client reports no error for an unreachable server ([bugs](#)).

Reading results

GUI and CLT reports come from one engine; `-regenerate` or the Archive view rebuilds them from stored data in html, pdf, csv, xls, xlsx, json or docx.

Latency CDF and CCDF. One line per flow on log axes; further left is lower latency. Where a line crosses P99, 99% of packets arrived within that latency. The plot stops at P99.99 ([article](#)).

Warning	Fix
Fewer than 50 data points	All packets fall in a narrow band. Narrow the histogram range in Preferences.
Packets below or above range	Move the range start or end in Preferences.
Fewer than 10,000 received packets	Send more packets or run longer.
Sampled packets	Expected on the 5100. On other models, update.

TCP graphs. A large gap between goodput and throughput means retransmissions. A steadily rising round-trip time means buffers filling. A drop in transmit window usually means loss ([diagnosis](#)).

Symptom	Likely cause	Fix
Throughput climbs slowly	Slow-start threshold too low	Set it to infinite
Stable but low throughput, no retransmissions	Window scale too low	Raise it; 4 to 6 is normally enough
Good throughput, round-trip time over 100 ms	Window scale too high	Lower it to 4 to 6

Symptom	Likely cause	Fix
Unstable throughput, many retransmissions	Packet loss	Rate-limit the flow, try Cubic (high latency) or SACK (Wi-Fi), capture and inspect

The right receive window is about the bandwidth-delay product, round-trip time times bandwidth.

TCP timing. Duration and throughput are always measured on the receiving side: for GET from first to last response segment at the client, for PUT from first to last request segment at the server. Reports also give time to first byte and minimum, average and maximum round-trip time.

Live metrics. Since GUI 2.18 the GUI serves OpenMetrics on port 8123; Prometheus scrapes it with a static target `localhost:8123`. Scenarios over 100 flows keep no over-time results in the GUI at all, so this is the only way to graph them ([metrics](#)).

Metric	Meaning	Labels
<code>byteblower_gui_traffic_bytes_total</code>	Bytes sent or received, reset per scenario; UDP and TCP	<code>flow</code> , <code>port</code> , <code>direction</code>
<code>byteblower_gui_latency_average_seconds</code> , <code>_minimum_</code> , <code>_maximum_</code> , <code>_jitter_</code>	One-way latency over the last second; frame blasting only	<code>flow</code> , <code>source</code> , <code>destination</code> , <code>latency_type</code>
<code>byteblower_gui_out_of_sequence</code>	Reordered packets, reset per scenario	<code>flow</code> , <code>port</code>

Machine-readable results. The JSON report lists `frameBlastingFlows[]` with `source.sent.packets` and `destinations[].received.packets`. The CLT is the scripted route to it:

```
ByteBlower-CLT -project clt_demo.bbp -scenario latency_under_load -output reports/
```

CLT exit code	Meaning
0	Success
64	Bad arguments
65	Project unreadable, usually saved by a newer GUI
66	Project, scenario or batch not found
70	Internal error
75	Temporary failure such as ARP; retry

The GUI must be closed while the CLT runs, and ports must already be docked in the project.

Troubleshooting

Most surprising results trace back to clocks, colliding filters, NAT or window settings.

Symptom	Cause	Fix
Negative latency	Receiver clock behind the sender's: two servers, or server and Endpoint, not continuously synced	PTP or NTP on both; prefer both ports on one server (article)
Latency spikes of seconds	Payload corruption altering the timestamp	Check loss alongside; fix the network
Negative loss (more received than sent)	Frames of two flows too similar for unique filters	Enable the Unique Frame Modifier, or change a length, payload byte or UDP port (article)
No traffic to a device behind a gateway	NAT not enabled on the port	Set NAT to Yes on the port
Endpoint cannot dock or receives nothing	Device has several IP addresses	Pick the traffic interface in the app; set NAT to Yes

Symptom	Cause	Fix
Endpoint shown as Unsupported	New OS version not yet verified	Usually works; update Meeting Point and Endpoint
100% loss with latency on IPv6 UDP	Timestamp breaking the UDP checksum on old versions	Update; software timestamping is now the default
Square-wave throughput on a 3100 or 3200	x710 NIC firmware flapping the link	Run the NIC firmware update on both NICs (article)
Server will not start: "Failed to allocate hostbuffers"	Kernel memory fragmentation	Reboot the server
1300: "No cores found on NUMA node 0"	BIOS lost NUMA setting	Re-save NUMA Support in BIOS, on site
Report missing graphs	Scenario has more than 100 flows	Export to Prometheus
Report too large or GUI out of memory	Over about 12,000 graphing-hours	Split scenarios, CSV only via CLT, or raise <code>-Xmx</code> in <code>ByteBlower.ini</code>
Upgrade wizard fails	Client cannot reach update servers or SSH	Update the GUI first; check proxy and port 22
Self-test loses frames	Cabling or load	Check link lights and cable; lower the load

To locate loss in time: read the over-time graph, then shorten the test, then split one flow into many short consecutive flows. To find which frames were lost, enable out-of-sequence, capture, and read the last 6 bytes of the payload as the frame number ([article](#)).

A port answers ping as soon as it has an address, which makes a five-line API script a quick reachability probe.

Hardware compatibility ([article](#)):

- 3100 and 3200 accept only Intel SFP+ modules.
- 4100: problems with Twinax DAC cables and with 1 Gbit/s SFPs; check NBASE-T SFP+ connectivity before relying on it.
- Netgear M4200, M4300 and Allied Telesis x950 switches: copper SFP+ transceivers either do not work or do not negotiate below 10 Gbit/s.

BPF filters select traffic for triggers and captures. Useful forms: `ip6 , udp port 80 , tcp dst port 80 , tcp[tcpflags] == tcp-syn , (tcp[tcpflags] & tcp-fin) > 0`. ECN-echo has no shorthand: `(tcp[tcpflags] & 0x40) > 0` ([cheatsheet](#)).

Coverage

Three of the four sites were crawled nearly completely; the Python reference was only partly readable, and the setup site not at all.

Site	Pages read	Gaps
Knowledge base	102 articles	About 70 article IDs need a login, errored or are gone. Release notes skipped on purpose. Many procedures exist only as screenshots.
Test Framework	76	None. Index and tag pages skipped as duplicates.
Tcl API	165, including all 127 class pages	Three source-file pages; the annotated higher-layer source.
Python API	5 doc pages, 15 example scripts	The reference page was readable only up to <code>ByteBlowerLicense</code> . Later classes are known by name and by example usage.
OpenMetrics	1	None
setup.byteblower.com	0	Blocks automated reading. Holds per-model installation manuals and downloads.

Pages were read through a tool that paraphrases prose, so commands, names and numbers here are as extracted but wording is not verbatim. The JSON examples and the Python example code came through unmodified.

Places where the docs contradict themselves:

- TCP window scale range: 0 to 8 in the GUI article and Tcl reference, 0 to 12 in the TCP example article and the Test Framework schema.

- Maximum frame size: 10000 bytes in the link-speed article, 8192 in the Tcl `Frame` reference.
- Endpoint to Meeting Point port: one article says 9002, the firewall table says 9100.
- Test Framework `report_prefix` default: `byteblower` in the reference, `report` in the quick start.
- Test Framework manual IPv6 key: `prefix_length` in the text, `prefix` in the schema.
- Test Framework VLAN `priority`: described as 3 bits, schema maximum is 3.
- NIC firmware article: says firmware "8.64 or older" but ships package 8.40.
- Result history depth: "5 seconds" on one Tcl page, 6 samples on another.
- Python capability examples in the docs contain syntax errors.

Not documented anywhere that was reachable: DOCSIS ATP pass thresholds, the TR-398 expected-throughput table, units for RFC 2544 `tolerated_frame_loss` and `accuracy`, and the latency noise floor per model (published only as an image).